# CSEN 11 Advanced Programming

Pointer Examples

# Pointer Examples

**Example 1: Modify a value using a pointer**

```
#include <stdio.h>

int main() {
    int x = 10;
    int *p = &x;

    *p = *p + 5;

    printf("%d\n", x);
    printf("%d\n", *p);
    return 0;
}
```

- p stores the **address** of x
- *p accesses/modifies the **actual value of x**
- Changing *p changes x

- Both x and *p prints out 15

# Pointer Examples

**Example 2: Passing a pointer to a function**

```
#include <stdio.h>

void increment(int *p) {
    *p = *p + 1;
}

int main() {
    int x = 10;
    increment(&x);
    printf("%d\n", x); //prints 11
    return 0;
}
```

- Passing the **address** of a variable to a function
- Modifying the original variable via dereferencing
- Difference between *call by value* and *call by address*

# Pointer Examples

**Example 3: Using two pointers in a function**

```
#include <stdio.h>

void add(int *a, int *b) {
    *a = *a + *b;
}

int main() {
    int x = 4, y = 6;

    add(&x, &y);

    printf("%d %d\n", x, y); //prints 10 6
    return 0;
}
```

- Passing **multiple pointers** to a function
- Dereferencing **both** pointers
- Modifying one variable based on another
- Demonstrates that pointers refer to **original variables**

# Pointer Examples

**Example 4: Increment all elements of an array**

```
#include <stdio.h>

void incrementArray(int *arr, int size) {
    int i;
    for (i = 0; i < size; i++) {
        *arr = *arr + 1;
        arr++;
    }
}
int main() {
    int a[4] = {1, 2, 3, 4};
    incrementArray(a, 4);
    for (int i = 0; i < 4; i++) {
        printf("%d ", a[i]);
    }
    return 0;
}
```

- Arrays **decay to pointers** when passed to functions
- int *arr receives the **base address** of the array
- Pointer arithmetic (arr++) moves to the next element
- Modifying *arr modifies the original array

Observe how we pass the array. We say a and not &a. This is because name of array is a pointer (address). &a is wrong.

Alternatively, incrementArray(&a[0],4) is valid.

# Pointer Examples

**Example 5: Traversing a string using a pointer**

```
#include <stdio.h>
int main() {
    char str[] = "CSEN";
    char *p = str;
    /* Using a pointer */
    while (*p != '\0') {
        printf("%c ", *p);
        p++;
    }
    printf("\n");
    /* Using the array name directly */
    int i = 0;
    while (str[i] != '\0') {
        printf("%c ", str[i]);
        i++;
    }
    return 0;
}
```

- A string is a **char array**
- The array name (str) decays to a pointer
- p walks through the string **character by character**
- '\0' marks the end of the string
- p++ moves to the next character
- *p accesses the current character

# How to print addresses

**Use %p and cast to (void *) when printing addresses**

```
int a = 5;
int *p = &a;

printf("%p %p\n", (void *)p, (void *)&a);
```

```
int *p = &x;

printf("%p\n", (void *)p);      // address stored in p
printf("%p\n", (void *)&p);     // address of the pointer variable
```

- %d → decimal integer
- %f → floating-point number
- %c → character
- %s → string
- **%p → pointer (address)**

# Common Pitfall

**An array name cannot be reassigned in C.**

```
char str[] = "CSEN";
char *p = str;
p = p + 1;      // VALID
printf("%s\n", p);    // prints SEN
```

p is a **pointer variable**
It stores an address
Pointer variables can be reassigned

```
char str[] = "CSEN";
printf("%s\n", str+1);    // prints SEN

str = str + 1;  // INVALID. compiler error
printf("%s\n", str);
```

str is an **array name**, not a pointer variable
str + 1 is a valid **expression -** *you can view a substring*

Arrays have a **fixed memory location**
Array names are **not assignable**
**str=str+1 is invalid**

# 2D arrays

```
#include <stdio.h>

int main() {
   int a[4][3] = {{1,  2,  3},{4,  5,  6},{7,  8,  9},{10, 11, 12} };

   /* Pointer to a row of 3 integers */
   int (*p)[3] = a;

   /* Access elements using pointers */
   printf("%d\n", *(*(p + 0) + 0));  // a[0][0] → 1
   printf("%d\n", *(*(p + 1) + 2));  // a[1][2] → 6
   printf("%d\n", *(*(p + 3) + 1));  // a[3][1] → 11

  printf("%d\n", *(*(a + 0) + 0));  // 1
  printf("%d\n", *(*(a + 1) + 2));  // 6
  printf("%d\n", *(*(a + 3) + 1));  // 11
   return 0;
}
```

```
1    2    3
4    5    6
7    8    9
10   11   12
```

- a decays to a pointer to its **first row**
- Type of a → int (*)[3]
- p + 1 → moves to the **next row** (skips 3 ints)
- *(p + i) → row i
- *(*(p + i) + j) → element a[i][j]

# 2D arrays

```c
#include <stdio.h>

int main() {
    int a[4][3] = {{1,  2,  3},{4,  5,  6},{7,  8,  9},{10, 11, 12} };

   /* Flatten the 2D array */
   int *n = &a[0][0];

   /* Access elements using flattened pointer */
   printf("%d\n", *(n + 0));   // a[0][0] -> 1
   printf("%d\n", *(n + 5));   // a[1][2] -> 6
   printf("%d\n", *(n + 10));  // a[3][1] -> 11

   /* Modify through flattened view */
   ++*n;                       // increments a[0][0]
   printf("%d\n", a[0][0]);    // now 2
   return 0;
}
```

**a sees the memory as rows of fixed size**

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 9 |
| 10 | 11 | 12 |

n sees the memory as a **single straight line of integers**

| Index: | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Value: | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
| | ^ | | | | | | | | | | | |
| | n | | | | | | | | | | | |